Original Paper

MEASUREMENT AND CONTROL

Measurement and Control
2019, Vol. 52(1-2) 100–115

DOI: 10.1177/0020294018819552
journals.sagepub.com/home/mac
SAGE

Automatic sorting system for industrial robot with 3D visual perception and natural language interaction

Yunhan Lin[1,2,3], Haotian Zhou[3], Mingyu Chen[3] and Huasong Min[3]

Abstract

With the development of the technologies such as computer vision, natural language interaction, process control technology, and sensor technology, the discussion of automatic sorting system of robot hits a hotspot in the robot community. This paper presents an automatic sorting system for industrial robot with the technologies of 3D visual perception, natural language interaction and automatic programming. Therein, a 'rule-scene' matching and interaction algorithm is proposed to combine all these modules together. By utilising our algorithm, robot can interact with human beings according to the real-time actual three-dimensional scene information and can guide users to give correct rules through speech when the rule is invalid. After getting the correct rules, robot can sort the object automatically using the automatic programming and execution algorithm designed in this study. In the experimental section, the designed system is applied to a fruit-sorting scene, which proves the effectiveness and practicability of the system.

Keywords

Automatic sorting system, human–robot interaction, human–robot–environment interaction, rule-scene matching algorithm, automatic programming

Date received: 12 July 2018; accepted: 25 November 2018

Introduction

With the development of the technologies such as computer vision, natural language interaction, process control technology, and sensor technology, the discussion of automatic sorting system of robot hits a hotspot in waste recycling domain[1] and parcel or product sorting domain[2,3] these years. Industrial robots could previously only complete automatic sorting using fixed programmes, which strengthens specificity while reduces expansibility of robots. As more and more diverse requirements for automatic sorting are put forward in industrial production, it becomes more and more difficult for previous industrial robots to meet requirements, so teaching-playback robots and robot programming languages have been developed.[4] Robot programming languages can be divided into action-level, object-level, and task-level programming languages according to the levels of operation description.[5] Among them, a task-level programming language refers to a language showing a higher level than the former two languages and is the ideal programming language. This language allows users to directly give commands to the goals to be achieved for a task, with no need for regulating the details of each action of robots. As long as the initial environment model and the final working state are given according to some principles, robots can undertake automatic reasoning and calculation and finally generate executable commands for robots automatically. Therefore, the key problem of designing the task-level automatic operating system for robots is to explore methods for acquiring operation commands and for realising automatic programming through task-level language use.

[1]College of Computer Science and Technology, Wuhan University of Science and Technology, Wuhan, China
[2]Hubei Province Key Laboratory of Intelligent Information Processing and Real-time Industrial System, Wuhan, China
[3]Institute of Robotics and Intelligent Systems, Wuhan University of Science and Technology, Wuhan, China

Corresponding author:
Huasong Min, Institute of Robotics and Intelligent Systems, Wuhan University of Science and Technology, No. 947, Heping Avenue, Wuhan 430081, Hubei, China.
Email: mhuasong@wust.edu.cn

Generally, operation commands are acquired through equipment input, document interaction with other software and natural language input. Choi et al.[6] developed the EL-E mobile robot which can automatically acquire and pass the target objects to users and the EL-E obtains the operation commands through the input using a laser pen and touch screen. By using the drawing exchange format (DXF) in automatic withdrawal method for a workpiece model, Xing et al.[7] analysed the operation information and then realised automatic programming for industrial robots based on virtual reality modelling language (VRML). Qin et al.[8] used the input information of industrial camera which obtains by workpiece images and corresponding QR code label to achieve automatic sorting of objects. Connell et al.[9] investigated the ELI robot and controlled its motion so as to get objects through natural language. This system can not only be used to execute pre-defined actions and recognise objects but also learn new nouns and verbs through speech training to construct a new semantic table. The system, designed by Fasola et al.,[10] enables the mobile robot PR2 to fulfil automatic grabbing tasks under the control of non-professional users through the use of natural language.

Comparison of different input methods for operation commands reveals that if the natural language can be used as the input of operation command to realise automatic sorting of robots, the robots can be more intelligent and easier to use. Meantime, natural language is the most natural and convenient way for human beings to communicate and obtain information; using natural language to control the robot will not need the user's ability of control knowledge and programming skills. In view of indoor mobile robot service requirements, Matuszek et al.[11] studied a natural language parser to realise automatic programming of robots. This system can translate the natural language describing the indoor path into LISP-like robot control language with which the robots can arrive at the destination in unknown indoor environments by analysing path commands. Based on the study of the natural Chinese language used to describe paths, Li and colleagues[12,13] designed, and realised, the visual navigation for mobile robots based on restricted natural language in indoor environment by establishing an intention map of navigation for mobile robots; moreover, they proposed the method for directly drawing movement paths of robots using natural language describing paths. All of the above automatic operating systems are designed for mobile service robots, while in the field of traditional industrial robots, automatic operating systems based on natural language are rarely seen. Actually, if they can be controlled by natural language, industrial robots can be operated by non-professional staff without special programming experience, which is revolutionary with regard to the improved usability of industrial robots. In addition, the above systems are used for automatic sorting when users give correct rules, but cannot be correctly operated when given wrong or invalid rules. This research proposes a 'rule-scene' matching and interaction algorithm by integrating the advanced technologies into the design of the automatic operating system for industrial robots. These technologies include the three-dimensional (3D) visual perception of robots, human–robot–environment interaction and automatic programming. By utilising our algorithm, robot can interact with human beings according to the real-time actual 3D scene information and can guide users to give correct rules through speech when the rule is invalid. After obtaining the correct rules, automatic sorting can be realised through the automatic programming and execution algorithm designed in this study.

Structure of the system

The designed automatic operating system for industrial robots mainly includes the 3D visual perception, the 'rule-scene' matching and interaction, and the automatic programming and execution modules. The system structure is shown in Figure 1. The 3D visual perception module provides high-quality semantic map information about the operating environment for the whole system. The 'rule-scene' matching and interaction module produces action of intention by combining rules obtained through human–robot interaction with real-time semantic map received through real-time 3D environmental perception. Furthermore, the automatic programming and execution module transforms action of intention with semantic information into executable commands for robots to control the manipulator to complete target actions. The hardware structure is based on our previous work[14], which includes three embedded computers, a RGB-D sensor, a speaker, a micphone and a modular manipulator with two finger end-effector.

3D visual perception

3D visual perception is the first step in the automatic sorting of intelligent industrial robots and provides high-quality semantic maps of the operating environment for the whole system. Generally, the feature descriptor algorithm for objects can be divided into global-based feature descriptors and local-based feature descriptors.[15] The recognition of global-based feature descriptors is difficult to use in multi-objective scenes which need to segment a scene into multiple clusters of objects. Moreover, due to the information of objects being lost due to occlusion, clutter, and changes of viewpoint, the calculated features are inaccurate, resulting in recognition errors. The local-based feature descriptors are used to characterise local 3D geometric information: the accuracy of local feature descriptor calculation is slightly influenced by object occlusion, clutter, and changes of viewpoint. The pipeline for 3D visual perception in this work consists of object

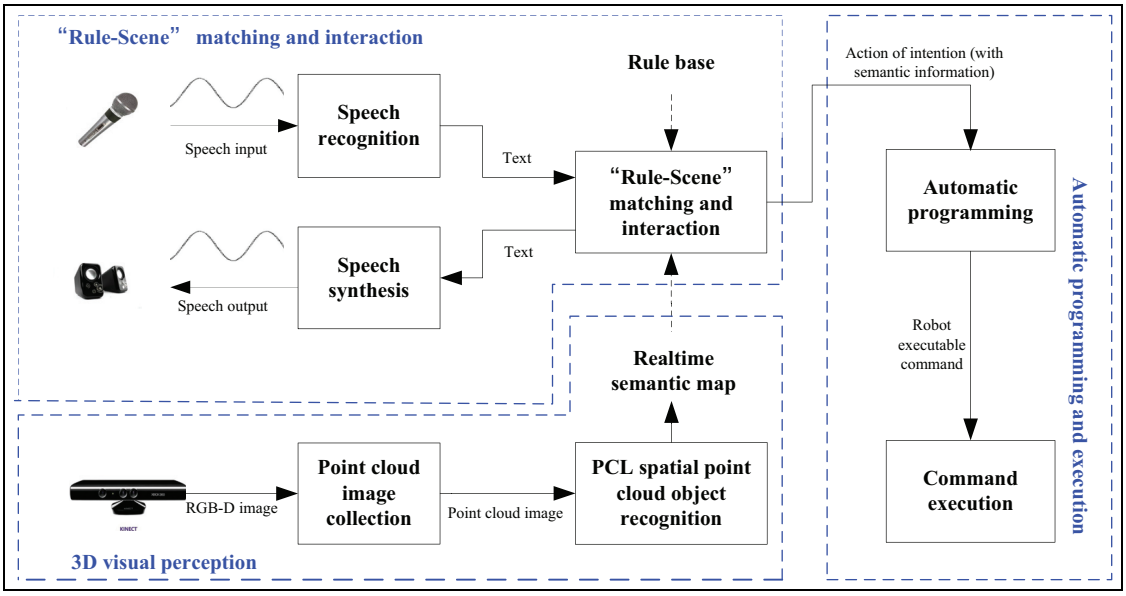


Figure 1. Structure of the designed automatic operating system for robots.

modelling, object recognition and pose estimation, and semantic map file generation[14] (Figure 2).

Object modelling. Object modelling is an offline process. First, the data are pre-processed, such as outlier removal, down-sampling, and upper-sampling for surface reconstruction, and then the objects are detected, which means single clusters are segmented from scenes for each perspective. Second, a suitable feature descriptor algorithm needs to be found to represent the significant characteristics on the surface of the objects. Alexandre et al.[16] tested the recognition accuracy of object categories and names and the performance of 1-Precision versus Recall curves of object recognition in a subset of a classification dataset[17] (48 objects in 10 classes) using the state-of-the-art descriptors. The tested descriptors include global feature descriptors, such as viewpoint feature histogram (VFH), clustered viewpoint feature histogram (CVFH) and ensemble of shape functions (ESF), as well as local feature descriptors, such as fast point feature histogram (FPFH), 3D Shape Context (3DSC), unique shape context(USC), signatures of histograms of orientations (SHOT) and colour signature of histograms of orientations (CSHOT). The experimental results show that the CSHOT is comprehensively optimal in recognising accuracy and time complexity.[18] Therefore, in this system, the CSHOT system is selected to describe the geometric features of point clouds in the vicinity of key points of intrinsic shape signatures (ISS).[19] Finally, the 4×4 transformation matrix of each frame is obtained using the checkerboard calibration algorithm, and the data with different perspectives are aligned and accumulated, thus obtaining 3D point clouds of objects of the model and artificially setting the identification of the object model.

Object recognition and pose estimation. During object recognition and pose estimation, a proper object model is allocated to each object in a scene by real-time acquisition of a frame of point cloud data from Kinect, which uses a local-based 3D recognition algorithm to describe the surface characteristics. Then, the transformation matrix from object model to corresponding objects in the scene is also obtained. In object recognition and pose estimation stage, the detection algorithm for key points and feature descriptor are consistent with those utilised in establishing the object model stage. After extracting ISS feature points in the scene and calculating CSHOT feature descriptors at key points, candidate models are generated through 3D feature matching based on distance threshold. The transformation hypotheses are generated using the random sample consensus algorithm and verified through iterative closest point algorithm, thus producing a solution scheme globally consistent with the scene.

Semantic map file generation. The generation of semantic map files writes object recognition data output into eXtensible Markup Language (XML) files to construct semantic map files of scenes. Each type of map information in the file is independent and new map information can be randomly added. Compared to the fixed form of relational database, the file shows good expansibility. Furthermore, XML files with tree structure can be used to demonstrate the relationship between data and attributes. The expression of a map file is as shown in Equation (1)

$$Object_i = \{category, name, colour, shape, x, y, z, size\} \quad (1)$$

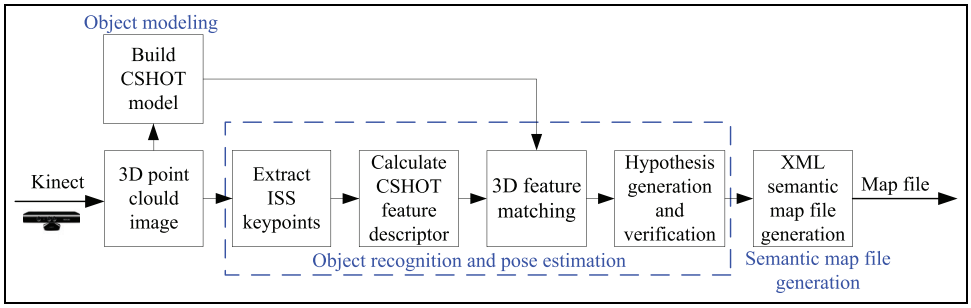


Figure 2. Pipeline for 3D visual perception.

where, $Object_i$, *category*, and *name* represent the object information in semantic maps of scenes, the object category, such as fruit and basket, and the object name, such as apple or orange, respectively. In addition, *colour, shape, x, y, z*, and *size* denote the object colour, shapes (cylinder, cube, etc.), the coordinate of objects in reference system of the manipulator, and the dimension of objects, that is length × width × height and $\pi\times$ (radius of bottom surface)$^2\times$ height, respectively.

'Rule-scene' matching and interaction

Figure 3 shows the flow of the 'rule-scene' matching and interaction algorithm. An open-source speech recognition system (PocketSphinx) for the embedded platform is used for speech recognition in the system,[20] and the multi-language and cross-platform open-source speech synthesis system (Ekho) is employed for speech synthesis.[21] The whole matching and interaction process mainly includes three functional modules: rule acquisition, 'rule-scene' matching, and guidance.

Rule acquisition. First, robots actively ask the users what they need to do. For example, robots ask 'Hello, I am xxx, what can I do for you?' Then, the natural language input of users is transformed into text through speech recognition and then the attributes of rules can be obtained based on lexical analysis, syntactical analysis, and semantic analysis. Finally, the rule attributes are saved into the rule base.

Lexical analysis. Since each different language has different lexical analysis processes, here, we choose Chinese as an example to explain the process of lexical analysis. Chinese lexical analysis comprises Chinese word segmentation, part of speech tagging, syntactic dependency analysis, and word sense disambiguation. A word is the minimum unit of language that can be used independently. Differing from English words separated using spaces, Chinese words are written continuously with a large character set. Therefore, Chinese words have to be segmented to divide a complete sentence in natural language into independent words, so that computers can obtain the definite boundary of Chinese words and understand semantic information contained in the text. The traditional methods for Chinese word segmentation include the matching methods based on dictionary, such as maximum forward matching, maximum reverse matching, and bidirectional matching[22]; however, due to the complexity of any language, there are a large number of words with ambiguous boundaries and unknown words. The matching methods merely based on dictionary cannot effectively solve key and difficult problems in the two aforementioned situations of Chinese word segmentation. Therefore, machine learning methods based on a large corpus have been widely used in recent years, and good results have been obtained. The statistical models used include the N-meta language model, the channel-noise model, maximum expectation, and the hidden Markov model. According to those actual situations facing the system, the NLPIR/ICTCLAS Chinese word segmentation system (2015 version)[23] developed by Dr Zhang Pinghua (Chinese Academy of Sciences) is used for this lexical analysis.

Syntactical analysis. Before carrying out syntax analysis, a syntax pre-processing stage is used: irrelevant modifiers in commands in natural language are eliminated. The pre-processed commands in natural language only comprise verbs, nouns, and modifiers relative to nouns. Syntax analysis uses the words in each lexical unit generated by the lexical analyser to construct the intermediate representation of the syntax tree. This intermediate representation provides the syntax structure of lexical cell flow generated from lexical analysis. Each internal node in the tree stands for a word and its child node represents its attribute. In view of the characteristics of commands in natural language in the system, the verbs that represent actions can be used as the central word of the whole command, while noun phrases are attached to the verbs and shown to be child nodes of actions in the syntax tree: the nouns of locality and adjectives for modifying nouns are subjected to noun phrases and expressed as child nodes of noun phrases in the syntax tree (Figure 4).

Semantic analysis. Semantic analysis is carried out to check whether, or not, the generated syntax tree

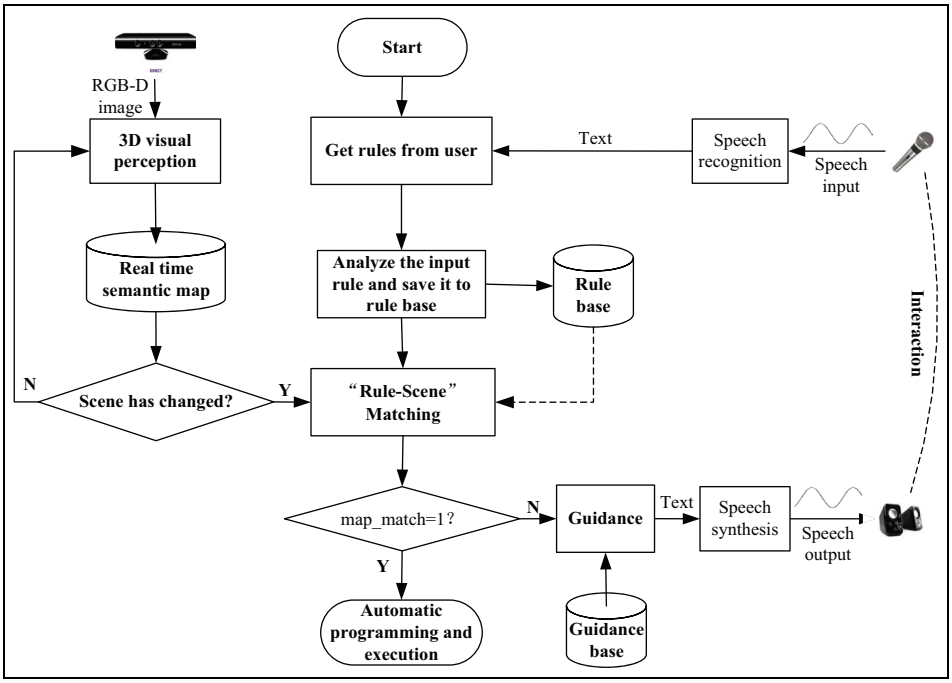


Figure 3. Flowchart for the 'rule-scene' matching and interaction algorithm.

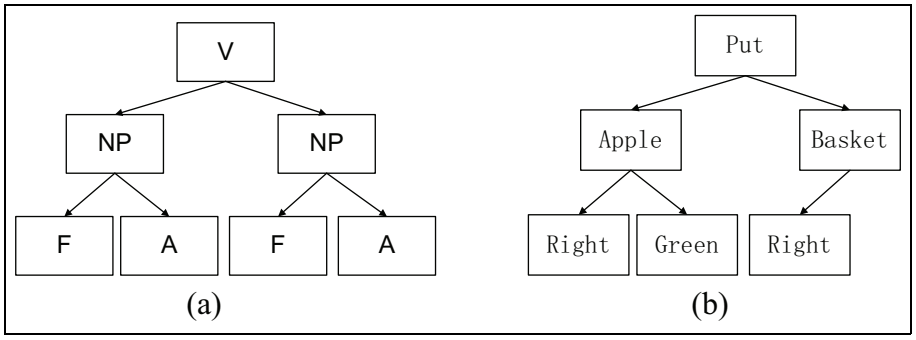


Figure 4. (a) Definition of the syntax tree structure and (b) an example of the syntax tree structure.
V, NP, F, and A represent the verb, noun phase, noun of locality, and adjective, respectively.

and semantic map are matched to examine whether, or not, commands in natural language are consistent with the semantic information defined by the language. The nodes in the syntax tress are matched with elements in the XML semantic map to obtain the operation sentence with semantic information, including designated actions and coordinates of target objects. The action requiring a robot to move an object from one place to another is defined as a composite action. The specific process is shown as follows: first, if the end-effector of robots is not in the same location as the target object, the end-effector is moved to this location and then the actions, such as capturing the object, moving to the target location, and putting down the object, are executed. The two actions, grabbing and putting down, belong to subsets of the composite action and only the two designated simple actions need to be executed.

Definition of rule base. A rule can be stored in a variety of forms, such as framework representation, object-oriented representation, and predicate representation.[24] In this paper, considering the requirement of our object sorting robot system, the format of framework representation is chosen to represent a rulebase. The rulebase can be described as formulas (2) and (3)

$$Rulebase = < Rule_1, Rule_2, \ldots, Rule_i > \quad (2)$$

$$Rule_i = < State, Solution > \quad (3)$$

here

State: The attributes set of a rule.

Solution: The solution of a rule (a set of actions).

Suppose that we want to grasp a target object at one place and put it into another place, the solution could be described as follows: move end-effector of manipulator to approach the target object which is vertical along

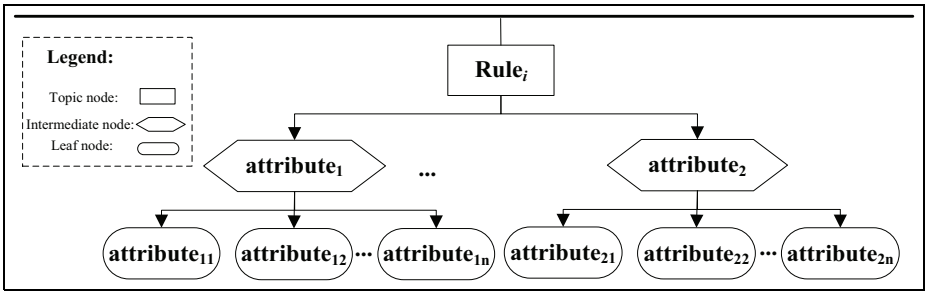


Figure 5. Storage structure of rule base.

z-axis at 100 mm (coordinate: 0.350, 0.100, 0.660); open grasper; end-effector arrives at the target point (coordinate: 0.350, 0.100, 0.560); close grasper; raise up end-effector (coordinate: 0.350, 0.100, 0.660); end-effector moves to another place (coordinate: 0.700, −0.300, 0.555); open grasper; back to original point (coordinate: 0.000, 0.000, 0.789); and close grasper. Each action of the solution above corresponds to a series of machine instruction which is used to control the robot.

By utilising a topic tree structure to describe each term of information in rules and relationship of information, the rule base is defined. There are three nodes in the topic tree: a topic node, an intermediate node, and a leaf node (Figure 5):

1. Each topic node represents the root of a topic tree and indicates the category of the rules and relevant knowledge base.
2. An intermediate node is a set of necessary attributes contained in the rules, that is, information required to make the rules effective.
3. A leaf node is used to save sub-attributes of intermediate nodes and is utilised as the supplementary attributes of intermediate nodes.

Each node has a binary effective state descriptor. The effective state descriptor of nodes represents whether, or not, the information about each node is effective, that is, whether, or not, it is confirmed by its users. Each state set has a corresponding dialogue generating function and the set of these dialogue generating functions constitutes the guidance base. In different system states, different response inputs can be obtained by calling this function and each dialogue generating function only responds to its corresponding state sets: they do not affect each other in the design and modification.

According to the above definitions, while designing an object sorting system, the attribute of the rule base can be defined as a topic tree including the names of those objects to be sorted and the names of placement destination thereof (Figure 6).

In Figure 6, *Obj_name, Obj_size, Obj_colour*, and *Obj_location* represent the name, size, colour, and location of the objects, respectively. *Des_name* and *Des_location* indicate the name and location of the placement destination of objects, respectively.

'Rule-scene' matching. In 'rule-scene' matching, the rules obtained using the system and real-time semantic map files of scenes generated in section '3D visual perception' are input to calculate whether, or not, rules match with scenes. In other words, the objects *expobj* referred to in the rules are sought in the scene semantic map and the matching formula of Equation (4) is used

$$map\_match = \begin{cases} 1 & expobj = object_i \cap expobj \\ 0 & expobj \neq object_i \cap expobj \end{cases} \quad (4)$$

where $object_i$ and $expob_j$ represent the object in a real-time scene and the object that is expected to be grasped in the rules, respectively.

When $map\_match = 1$, it indicates that there are expected objects in the rules in the scenes. While $map\_match = 0$, it shows that there are no expected object in the rules in the scenes, thus it is judged that the rules are invalid in the current environment. At this time, the guidance mode of the system has started to inform users of the object situations in the current scene and ask users to provide effective expectations.

Guidance. As described in the definition of the storage structure of rule base in section 'Rule acquisition,' each node has a corresponding dialogue generating function and the set of these dialogue generating functions constitutes the guidance base *GuidanceBase*. In accordance with the state set *node_state* of binary effective state descriptors of all nodes in the topic tree, the system calls the guidance solution *guidance_solution*. The definition of *GuidanceBase* is given in Equations (5) and (6)

$$GuidanceBase = (GC_1, GC_2, \ldots, GC_n) \quad (5)$$

$$GC_i = (node\_state_i, guidance\_solution_i) \quad (6)$$

Automatic programming and execution

Figure 7 shows the automatic programming and execution process. Real-time semantic map files in

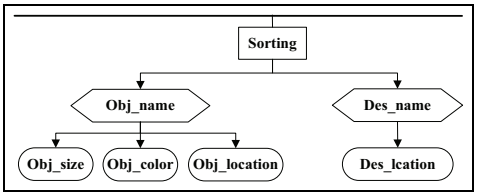


Figure 6. Storage structure of rule base of object sorting system.

scenes obtained by combining action of intention with semantic information obtained after rule-scene matching and interaction with 3D environmental perception are used as the input of automatic programming for the robot. A series of executable commands that can be understood by the robot are transformed through intention interpretation, code parsing, and code compiling. Command execution indicates that the robot executable commands are transferred to the rotation angle of each joint of the robot through the communication bus to control actual movements of the manipulator.

Intention interpretation. Intention interpretation means that the action of intention with semantic information is transformed into a series of programme codes for the robot through robot programming language. By comparing some international, state-of-the-art programming languages, such as INFORM, RAPID, KRL, and URScript, this study designs a language set for our modular manipulator in accordance with the features of code types, syntax rules, and analysis method. This language set consists of variable, state, control, movement, and operation commands (as shown in Table 1).

The pseudo-codes of intention interpretation are shown in Algorithm 1. First, the operation rules are sought according to the relevant attributes of objects in the scene to generate programme templates. Then passing points are obtained in accordance with the coordinates of objects to be sorted and destination for placement, and then, the tracks of each passing points are planned. Finally, the instantiated programme is generated and the instantiated programmes of all objects are collected to constitute robot language source codes. Figure 8 shows the programme code with which the robot picks an object at point A and then places it at point B. It is assumed that the coordinates of points A and B are (−0.080, 0.331, 1.350) and (0.486, 0.372, 1.230).

Word segmentation and matching. Before designing the parser, the word segmentation system of languages for the robot is first constructed. Due to the robot language source codes used in this study being English words, a complete robot sentence contains more than one word and these words are separated by spaces according to the rules of English grammar. Therefore, two enumerated word libraries are defined in the header file. One of the word libraries covers all previous commands for the robot. The pseudo-codes are given below.

The first word library consists of some state words in word segmentation and the pseudo-codes are as follows:

```
typedef enum
{START, INID, INNUM, DONE, INPOINT // defining
word library of the state words in word segmentation
}StateType;
```

The second word library contains not only all codes for the robot but also special characters in the language text file. These characters include carriage return, terminator, data volume, and undefined words. The pseudo-codes are as follows:

```
typedef enum
{ENDFILE, ENDLINE, INVALID, INIT, BEGIN,
TIMER, HAND, ON, OFF, MOVEP, MOVEL, MOVEJ,
PAUSE, END, B, I, P, J, X, Y, Z, SET, CLEAR,
CREAT, ADD, SUB, INC, DEC, ID, NUM // defining
an enumerated word library covering all codes for the
robot
}Tokentype;
```

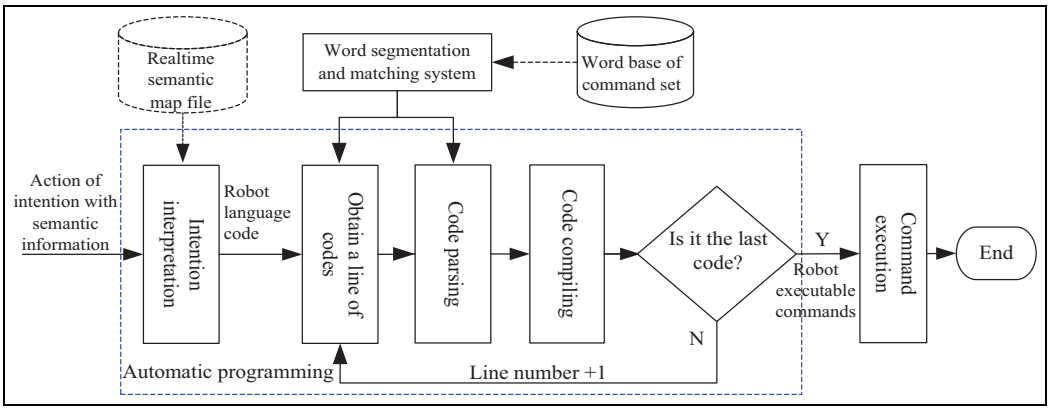


Figure 7. Automatic programming and execution process.

Algorithm 1. Pseudo-codes of intention interpretation.

Input: SI ∩ RSM //SI (Semantic Intention), RSM (Real-time Semantic Map)
Output: RLSC //RLSC (Robot Language Source Codes)
for i = 0, i < N, i + + //The total number of objects in the scene is N
  The operation rules ($rule_i$, $attribute_1$, $attribute_2$) are sought according to the relevant attributes of
  $Object_i = \{category, name, colour, shape, x, y, z, size\}$
  Generating the current operation programme template
  $P_{pick}\leftarrow$ object coordinate of $attribute_1$
  $P_{place}\leftarrow$ object coordinate of $attribute_2$
  Calculating passing points P_0, P_1, ..., P_{pick}, ..., P_{place}, ..., P_m, ..., P_0 according to P_{pick} and P_{place} //P_0 indicates the location of the manipulator at initial status.
  Planning trajectory of each path
  Generating instantiated programme
end for
Generating RLSC

Table 1. Robot language set of the manipulator.

Command type	Example	Note
Variable command	SET B_0 c	Set B_0 = 'c'
	CREAT P_1 *x y z*	Create point variable P_1 and assign its coordinates to (*x, y, z*)
	CREAT J_1 a b c d e f	Create joint variable J_1 and assign the values of each joint to a, b, c, d, e, and f
State command	INIT	Initialise manipulator the initial state
	BEGIN	Move the manipulator to the preparation state
	BEGIN J_1	Set the state of J_1 to be the preparation state and move it to this state
Control command	CLEAR P_1	Clear the variable P_1
	INC I_1	Add 1 to the I_1
	DEC I_1	Subtract 1 from the I_1
	ADD I_2 5	Add 5 to the value of variable I_2
	ADD P_1*x* 30	Increase the *x*-coordinate of point P_1 by 30
	SUB I_2 5	Subtract 5 from the value of variable I_2
	SUB P_1*x* 30	Subtract 30 from the *x*-coordinate of point P_1
	TIMER 3	The manipulator waits for 3 s in its current state
	PAUSE	The manipulator pauses in its current state
Movement command	MOVEP P_1	Move to point P_1 in the form of curve interpolation
	MOVEL P_1	Move to point P_1 in the form of linear interpolation
	MOVEJ 4 45	Rotate the no. 4 joint axis of the manipulator to 45°
Operation command	HAND ON	Turn on end-effector
	HAND OFF	Turn off end-effector

After defining the above two word libraries in the header file, word segmentation and matching is carried out. There are two steps in our word segmentation and matching process. The overall process of word segmentation and matching function is shown in Figure 9:

Step 1. Using the first letter of a word to preliminarily distinguish the state of an instruction. The variable name of C language can only be started by letter, number, and underline. Here, START and DONE means the start and the end character of a word, respectively; INID means the character is a letter; INNUM means the character is a number; and INPOINT means the character is a underline.
Step 2. Classifying the word into the type of Tokentype. Please note that the word INVALID is the error state. The input of the whole word segmentation and matching function is empty, while the output is the enumeration type of Tokentype. Before constructing the function, a character array *lineBuf* is first defined to store a line of texts and a character array *tokenString* is defined to store the first code after word segmentation.

Code parsing. Figure 10 shows the process of analytical function of our robot language parser: because the previous main function has already acquired a line of robot codes, the analytical function first calls the word segmentation and matching function to obtain the first word. The overall frame of the function is based on a 'switch-case' structure. Each word is defined as a case. The variables are calculated using the analytical

```
% Manipulator is used to pick the object at point A (-0.080, 0.331, 1.350)
and place it at point B (0.486, 0.372, 1.230)
INIT          % initialize state of the manipulator
BEGIN         % move the manipulator to the preparation state
MOVEP P1      % move to point P1 (-0.080, 0.331, 1.450), which is 0.1 m
above point A in the direction of the z-axis
HANDON        % turn on end-effector
TIMER 1       % manipulator waits for 1 s in its current state
MOVEL P2      % (P2) move to point A (P2)
HANDOFF       % turn off end-effector
TIMER 1       % manipulator waits for 1 s in its current state
MOVEL P1      % return back to point P1
MOVEL P3      % move to point P3 (0.486, 0.372, 1.330), which is 0.1 m
above point B in the direction of the z-axis
MOVEL P4      % move to point B (P4)
HANDON        % turn on end-effector
TIMER 1       % manipulator waits for 1 s in its current state
MOVEL P3      % move to point P3
BEGIN         % move the manipulator to the preparation state
```

Figure 8. Robot language source code with which the robot picks an object at A and then places it at B.

function according to the syntactical rules of the robot language set (defined previously). For example, when the first command is segmented and matched to be MOVEP, the coordinates of the objective point p of MOVEP are (p_x, p_y, p_z) and are obtained by calling the analytical function for joint interpolation and movement. The inverse solutions of objective points are calculated by utilising the calculation method of inverse kinematics. Furthermore, in accordance with the resulting inverse solutions, and the angle of the current location, an analytical matrix is interpolated. This matrix has N rows and six columns, where N represents the number of interpolation points and six parameters in each row indicate the degree of rotation of each joint. After the movement parsing of MOVEP, the analytical matrix thus obtained is saved in the compiling matrix and the current position, pose, and rotation of the manipulator are saved.

The operational commands, such as INC, DEC, ADD, and SUB, are omitted from the figure. When the first code is segmented and matched to be operation command, the corresponding analytical function is invoked to calculate the variables in accordance with the previous grammar rules of the robot language set.

Code compiling. Figure 11 shows the process of running the compiling functions of our robot language parser. The input of compiling functions is the returned value of analytical functions and the structure of compiling functions is similar with that of analytical functions, belonging to the selected programme structure. Due to compiling functions being based on executable commands for the manipulator, the compiling functions are not used to deal with variable commands, such as SET and CREAT and operation commands, such as INC, DEC, ADD, and SUB (omitted from the figure). Therefore, the compiling functions are simpler than the analytical functions.

The compiling functions mainly aimed at state commands (INIT and BEGIN) and movement commands (MOVE), as well as commands such as TIMER and HAND. By taking MOVEP as an example, when the obtained returned value belongs to the MOVEP command, the compiling matrix in the analytical function is invoked and each joint degree in the compiling matrix is compiled into the CAN command and saved into the execution matrix.

Command execution. After analysing and compiling each line of robot codes, the execution matrix is obtained. Each line of elements in this matrix is composed of executable commands and check-codes are added in the compiling functions for each line of elements, that is, the first element represents the check. Therefore, the execution functions can be utilised to execute commands in the matrix, without error, by merely judging the previous check codes.

The process of the execution function is shown in Figure 12. A line of elements is first obtained and the first element is checked using check rules defined by the compiling functions. The check results show what type of elements (commands) is in this line and the corresponding CAN, timer, and end-effector commands can be executed according to the check. Finally, if there is an undefined check character, the function reports an error.

Now, we take the application of fruit sorting as an example to explain the whole process of our designed automatic operating system. Assume that there are three red apples and two green apples on the platform and two baskets to the left and right of the robot are used to place the sorted fruit.

[Robot]: Hi, I am WUSTER, what can I do for you? (This sentence is pre-set in the system and is used to guide the user to give initial operation rules.)

[User]: Sort fruit. (The rule is unclear, because users only provide an object–fruit while the location where the objects are placed in is unclear. As shown in Figure 6, only one in

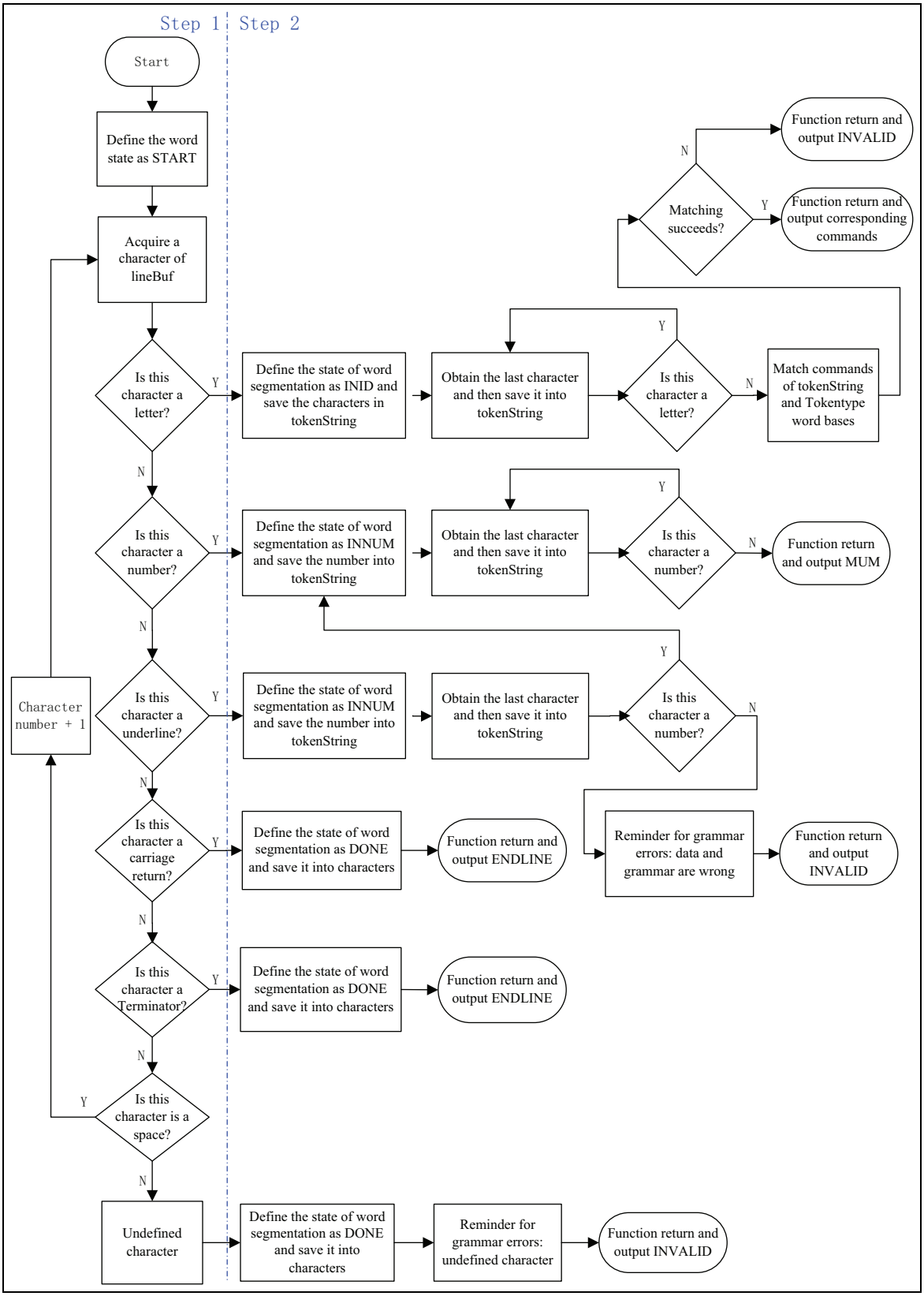


Figure 9. Process of the word segmentation and matching function.

the two attributes of the intermediate node is matched, so the rule is incomplete. The robot reports the object situation in the scene to the user according to that real-time object information in the scene, so as to guide the user to give effective rules.)

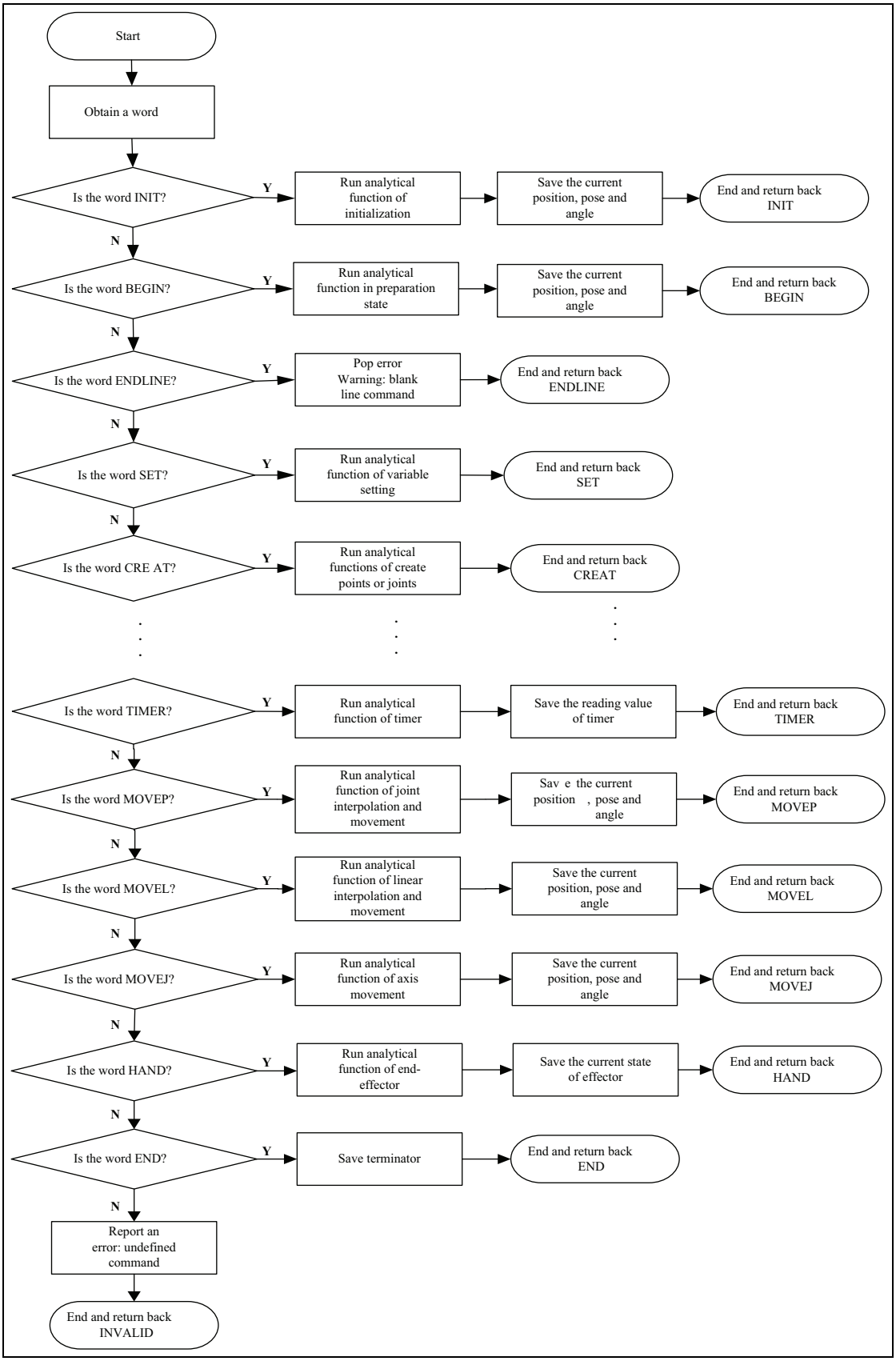


Figure 10. Process of the analytical function.

[Robot]: Here are three red apples, two green apples, and two baskets to the left and right, please give the rules. (The robot informs the user of the object situation in the current scene to guide the user to present rules.)

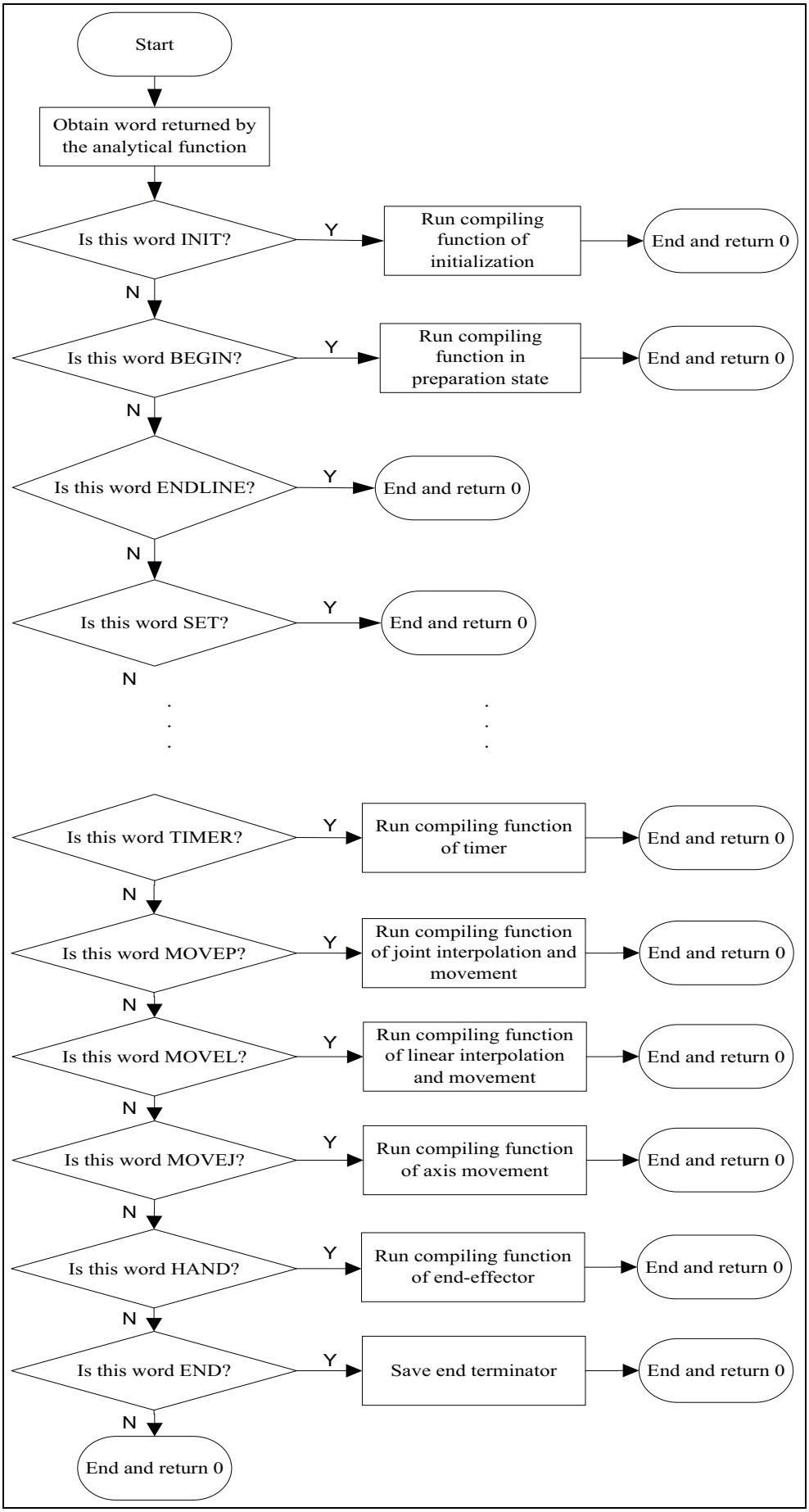


Figure 11. Process of the compiling function.

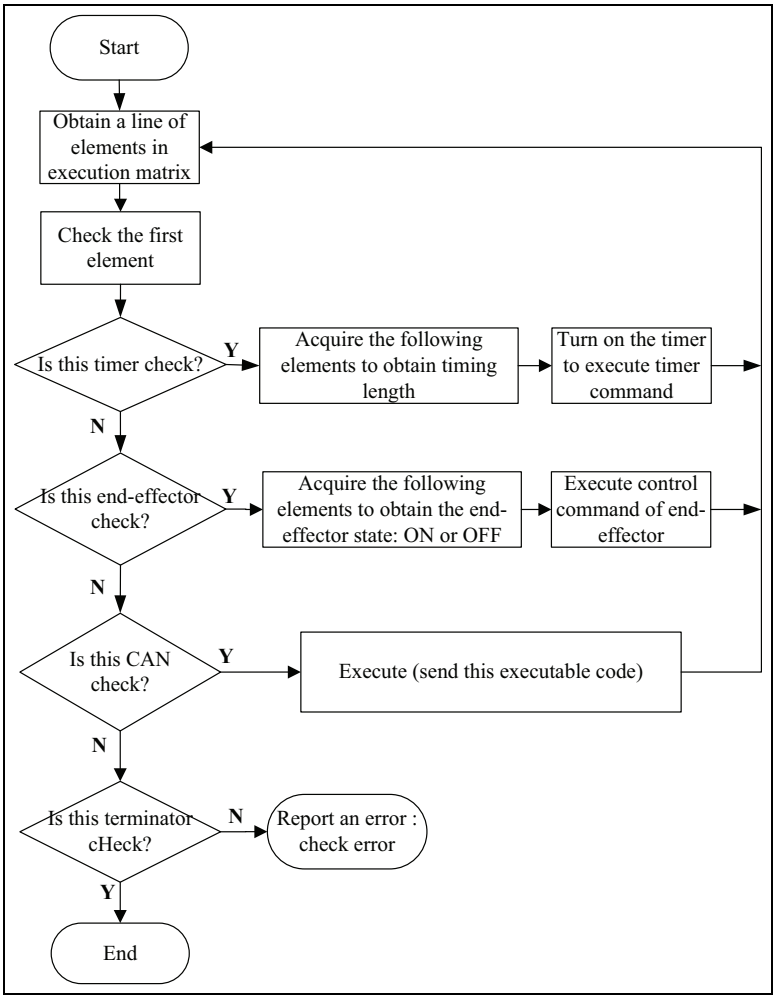


Figure 12. The execution function process.

[User]: Place all red apples in the left basket. (This rule is clear, because two attributes of the intermediate node are defined clearly and *map_match* = 1 is obtained by matching the scene and the expected objects, that is, this rule is effective and can be executed.)

[Robot]: Okay. (After response, the robot executes the command.)

[Robot]: Here are still two green apples, in which basket should they be placed? (After sorting red apples, the robot finds the rule of placing all red apples in the left basket does not match with the scene and *map_match* = 0 through matching the rule and the scene. The guidance mode is then started and the robot reports the object situation in the current scene and guides the user to define new rules.)

[User]: Put all green apples in the right basket. (This rule is clear, because there are clear definitions of two attributes of the intermediate node and *map_match* = 1 is obtained by matching the scene and the expected objects.)

[Robot]: Okay. (After response, the robot executes this task.)

[Robot]: Sorting is completed. (After sorting, the robot finds that there is no object to be sorted by matching the rule and the scene and then reports that the sorting process is completed.)

Experiment and analysis

In the experimental section, an application scene for sorting fruit using the Chinese speech control system is designed to verify the effectiveness and practicability of the system. The main body of the manipulator used in the experiment is WUST-ARM modular manipulator and each function module is run on an Ubuntu (version 12.04) desktop computer with Intel Core™ i5 CPU 650 at 3.20 GHz × 4 with 8GB RAM. Moreover, the Kinect is fixed in front of the WUST-ARM, as shown in Figure 13. There are 12 fruits (from five species) in the dataset of the experimental platform. Of them, red apples include big and small ones and carambola has big and small varieties too, while other fruits are the same size but of different colours (Table 2).

Time test for 3D scene recognition

Considering that there is limited working space for the WUST-ARM and it is unfavourable for capturing and recognition if the objects are placed too close together, this experiment divides all types of fruits in the dataset into two batches to be placed on a production line and each group is tested 10 times, so as to count and analyse the average time for object recognition. As shown in Table 3, it can be seen that the time for collecting data in the scene is 1.8 s, accounting for about half of the total time of 3.7 s needed for object recognition. This is because the current scene has a complex background and the Kinect needs to collect 307,200 points in the scene, in real-time, and stores them as a variable for subsequent recognition.

Test of accuracy of object recognition in the 3D scene

In this experiment, four datasets are set to compare the accuracy of object recognition in the 3D scene. The datasets are as follows:

Dataset A: an object is placed on the conveyor;
Dataset B: three objects are placed on the conveyor;
Dataset C: five objects are placed on the conveyor;
Dataset D: seven objects are placed on the conveyor.

Each group of experiment is repeated 100 times with different placement positions and poses of objects. The average time for object recognition, accurate recognition rate, missed recognition rate, and false recognition rate are obtained, and the root mean square error (RMSE) for the 3D coordinate values and size of objects are calculated. The RMSE is given by

$$RMSE = \frac{1}{4}\sum_{j=1}^{4}\sqrt{\frac{\sum_{i=1}^{n}\left(X^{j}_{obj,\,i} - X^{j}_{model,\,i}\right)^{2}}{n}} \qquad (7)$$

where j is the number of elements and i represents the number of objects to be recognised in the scene.

As demonstrated in Table 4, in the verification scene of a small-scale model base, with the increase in the number of objects, the processing time for object recognition increases significantly. This is because the number of feature points of scenes, normal calculations, and subsequent cluster segmentation operations increase, so that classified storage takes more time. As for recognition accuracy, with the increasing number of objects in scenes, the accuracy decreases slightly and the RMSE of object recognition accuracy stabilises at about 4 mm.

Actually, the accuracy of object recognition based on 3D point cloud depends on the occlusion rate of target objects and clutter rate of objects in scenes. In 2015, Guo Yulan verified the recognition rate of some of the state-of-the-art local feature descriptors (including the CSHOT feature descriptor used in this work) on multiple datasets. The experiments show that, with the increase in occlusion rate of objects and clutter rate of scenes, the recognition rate of objects decreases.[25] To verify the effectiveness of the whole system, the verification is only carried out on small-scale model bases. The model base is small and geometric features and descriptors are selected properly. Moreover, the objects are placed without occlusion and clutter. For these reasons, the system shows good recognition effects and the accuracy of semantic map information can meet operational demands in actual experimental verification trials.

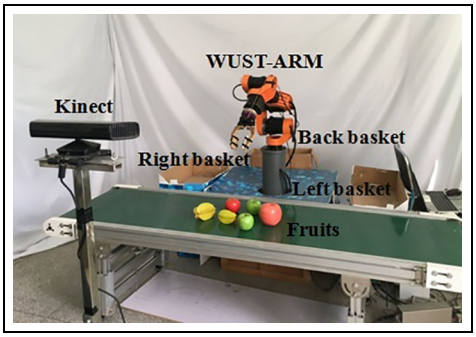


Figure 13. Experimental platform of WUST-ARM fruit-sorting system.

Table 2. Species of fruit used.

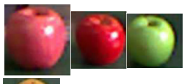
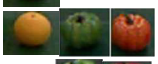
Species	No.	Example
Apple	3	
Orange	1	
Pumpkin	3	
Coloured sweet pepper	3	
Carambola	2	

Table 3. Processing time for object recognition in a 3D scene.

No.	Step	Average time (ms)
1	Scene collection	1822
2	Filter pre-processing	64.1
3	Feature extraction	290.2
4	Feature matching, hypothesis verification, and classified storage	1468
Total time		3644.3

Accuracy test for fruit sorting

In this experiment, the three types of fruits to be sorted were placed on the production line, including red apples, green apples, and carambola: three

Table 4. Comparisons of experimental data in different experimental scenes.

Test dataset	Average time for object recognition (ms)	Accurate recognition rate (%)	Missed recognition rate (%)	False recognition rate (%)	Average value of RMSE (mm)
A	2617	100.0	0.0	0.0	2.8
B	2964	96.0	0.0	4.0	4.6
C	3352	95.6	0.0	4.4	3.6
D	3740	94.7	0.0	5.3	3.9

baskets were put on the left, right, and back of the WUST-ARM. Users determined rules through interaction with the system in Chinese natural language. In this experiment, users, including five males and five females, were invited to interact individually with the robot 10 times, giving a total of 100 tests. Before testing, a large number of training and testing runs were conducted on the open-source speech recognition system PocketSphinx and the speech synthesis system Ekho on a few, limited test-sets of natural language, which realised 100% accurate speech recognition and synthesis of test statements. Therefore, there is no error of speech recognition and synthesis in this test. The test data for fruit sorting are shown in Table 5. It can be seen from the table that the rule acquisition rate of the 'scene-rule' matching algorithm is 100% after eliminating errors of speech recognition and the correct sorting rate of objects are equivalent to the recognition rate of objects listed in Table 4. This proves the effectiveness and practicability of the automatic operating system.

Demonstration

Now, a specific demonstration is designed to sort fruits on the flow production line. The process of whole demonstration is a consecutive process which includes three scenes:

1. *Scene 1: Primary rule definition*. In this scene, two types of objects are located on the production line, rules are defined by human–robot–environment interaction, and system automatically sorts fruit following the defined rules.
2. *Scene 2: Encounter new object*. Based on scene 1, in this scene, a new type of object is added into this scene. First, our system sorts the fruits which are matched with defined rules and leaves the new type of fruit on the production line. Then, the robot requests new rule from the user. After getting the rule, system automatically sorts the left fruits.
3. *Scene 3: Automatic operation under defined rules*. In this scene, a batch of fruits move to the front of the robot, be different from scene 2, at this time, all of the fruits are matched with the rules, then system automatically sorts fruits following the defined rules.

Table 5. Comparisons of experimental data for fruit sorting.

User	Acquisition accuracy of rules (%)	Sorting accuracy of objects (%)
Male	100	94
Female	100	96

The actual demonstration video is available in the following website: https://v.youku.com/v_show/id_XMzcxNjQxOTk4MA==.html?spm=a2h3j.8428770.3416059.1

Conclusion and future work

An automatic operating system for a robot, based on interaction with Chinese natural language and integrating advanced technologies, such as 3D visual perception, human–robot–environment interaction, and automatic programming, was designed for use in the automatic operating system of a modular industrial manipulator. In the experiments, the recognition time for 3D scenes, object recognition accuracy, and fruit-sorting accuracy are tested. The test results demonstrate that (1) the 'rule-scene' matching and interaction algorithm designed in this study is effective and correct and can correctly guide users to provide correct rules; (2) the sorting accuracy of objects depends on the recognition rate of objects; and (3) the system enables the manipulator to sort objects automatically under the control of natural language.

For future works, our improvement efforts will focus on the research of the feature descriptor algorithm of 3D visual perception and finally to improve the accuracy of object recognition in our automatic operating system. Meantime, we will expand the robot language command set in Table 1 to meet more complex situations, including more complete robot control language set and more complex scenarios.

Declaration of conflicting interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Funding

This work was supported by the National Key R&D programme of China (grant no. 2017YFB1300400) and the

National Natural Science Foundation of China (grant no. 61673304).

ORCID iD

Yunhan Lin https://orcid.org/0000-0003-2779-2637

References

1. Gundupalli SP, Hait S and Thakur A. A review on automated sorting of source-separated municipal solid waste for recycling. *Waste Manag* 2017; 60: 56–74.
2. Singh AK and Ali MSR. Automatic sorting of object by their colour and dimension with speed or process control of induction motor. In: *IEEE international conference on circuit, power and computing technologies (ICCPCT)*, Kollam, India, 20–21 April 2017, pp. 1–6. New York: IEEE.
3. Shah R and Pandey AB. Concept for automated sorting robotic arm. *Proc Manuf* 2018; 20: 400–405.
4. Liu K, Li SZ and Wang BX. Research of the direct teaching system based on Universal Robot. *Sci Technol Eng* 2015; 15(28): 22–26.
5. Cai ZX. *Robotics*. 2th ed. Beijing, China: Tsinghua University Press, 2009, p. 285.
6. Choi YS, Chen T, Jain A, et al. Hand it over or set it down: a user study of object delivery with an assistive mobile manipulator. In: *18th IEEE international symposium on robot and human interactive communication*, Toyama, Japan, 27 September–2 October 2009, pp. 736–743. New York: IEEE.
7. Xing JS, Gan YH and Dai XZ. Auto-programming system based on the workpiece model for industrial robot. *Robot* 2017; 39(1): 111–118.
8. Qin Q, Zhu D, Tu Z, et al. Sorting system of robot based on vision detection. In: *International Workshop of advanced manufacturing and automation*, Singapore, 22–23 July 2017, pp. 591–597. New York: Springer.
9. Connell J, Marcheret E, Pankanti S, et al. An extensible language interface for robot manipulation. In: *International conference on artificial general intelligence*, Oxford, 8–11 December 2012, pp. 21–30. New York: Springer.
10. Fasola J and Mataric MJ. Interpreting instruction sequences in spatial language discourse with pragmatics towards natural human-robot interaction. In: *IEEE international conference on robotics and automation (ICRA)*, Hong Kong, China, 31 May–5 June 2014, pp. 2720–2727. New York: IEEE.
11. Matuszek C, Herbst E, Zettlemoyer L, et al. Learning to parse natural language commands to a robot control system. In: *International symposium on experimental robotics*, Montreal, QC, Canada, 18–21 January 2012, pp. 403–415. New York: Springer.
12. Li XD, Zhang XL and Dai XZ. A visual navigation method of mobile robot based on constrained natural language processing. *Robot* 2011; 33(6): 724–749.
13. Li XD and Zhang XL. A route instruction method using natural language processing for indoor intelligent robot navigation. *Acta Automatica Sin* 2014; 40(2): 289–305.
14. Lin YH, Min HS, Zhou HT and Pei FL. A human-robot-environment interactive reasoning mechanism for object sorting robot. IEEE Transactions on Cognitive and Developmental Systems. 2018, 10(3): 611–623.
15. Aldoma A, Marton ZC, Tombari F, et al. Tutorial: point cloud library: three-dimensional object recognition and 6 DOF pose estimation. *IEEE Robotics & Automation Magazine* 2012; 19(3): 80–91.
16. Alexandre LA. 3D descriptors for object and category recognition: a comparative evaluation. In: *Workshop on color-depth camera fusion in robotics at IEEE international conference on intelligent robots and systems (IROS)*, Vilamoura, Portugal, 7–12 October 2012, pp. 1–7. New York: IEEE.
17. Lai K, Bo L, Ren X, et al. A large-scale hierarchical multi-view RGB-D object dataset. In: *IEEE international conference on robotics and automation (ICRA)*, Shanghai, China, 9–13 May 2011, pp. 1817–1824. New York: IEEE.
18. Tombari F, Salti S and Di Stefano L. A combined texture-shape descriptor for enhanced 3D feature matching. In: *18th IEEE international conference on image processing (ICIP)*, Brussels, 11–14 September 2011, pp. 809–812. New York: IEEE.
19. Zhong Y. Intrinsic shape signatures: a shape descriptor for 3D object recognition. In *12th international conference on computer vision workshops (ICCV)*, Kyoto, Japan, 29 September–2 October 2009, pp. 689–696. New York: IEEE.
20. Huggins-Daines D, Kumar M, Chan A, et al. PocketSphinx: a free, real-time continuous speech recognition system for hand-held devices. In: *International conference on acoustics speech and signal processing (ICASSP)*, Toulouse, 14–19 May 2006, pp. 185–188. New York: IEEE.
21. Wong GN. Ekho – Chinese text-to-speech software, http://www.eguidedog.net (2016, accessed 25 March 2018).
22. Peng Q and Yu CQ. A brief analysis the Chinese word segmentation method. *Infor Comm* 2015; 147(3): 92–93.
23. Zhang HP. NLPIR/ICTCLAS Chinese word segmentation system, http://ictclas.nlpir.org/ (2015, accessed 25 March 2018).
24. Shiu SCK and Pal S. Case-based reasoning: concepts, features and soft computing. *Appl Intell* 2004; 21(3): 233–238.
25. Guo YL, Bennamoun M, Sohel F, et al. A comprehensive performance evaluation of 3D local feature descriptors. *Int J Comput Vision* 2016; 116(1): 66–89.